

LAB # 5

Inter-Thread Communication:

OBJECTIVE

Develop an inter-thread user communication program by using synchronization.

Lab Task:

1. Design a simple program of concurrency by implementing the scenario of two account holders in a joint bank account. (Hint: Total amount will be 50000, if 'user A' wants to withdraw 45,000 and 'user B' wants to withdraw 20,000) Apply mechanism of synchronization e.g. Block or Method for handling accessibility of multi-threads:

```
package SDA;

class JointAccount {
    private int balance = 50000;

    synchronized void withdraw(String user, int amount) {
        System.out.println(user + " is trying to withdraw " + amount);

        if (balance >= amount) {
            System.out.println(user + " is proceeding with withdrawal...");
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                System.out.println(e);
            }

            balance -= amount;
            System.out.println(user + " successfully withdrew " + amount);
            System.out.println("Remaining Balance: " + balance);
        } else {
            System.out.println("Insufficient balance for " + user + " to withdraw " + amount);
            System.out.println("Remaining Balance: " + balance);
        }
    }
}

class User extends Thread {
    JointAccount account;
    String userName;
    int amount;

    class User extends Thread {
        JointAccount account;
        String userName;
        int amount;

        User(JointAccount account, String userName, int amount) {
            this.account = account;
            this.userName = userName;
            this.amount = amount;
        }

        @Override
        public void run() {
            account.withdraw(userName, amount);
        }
    }
}

public class Lab {
    public static void main(String[] args) {
        JointAccount acc = new JointAccount();

        User userA = new User(acc, "Affan", 45000);
        User userB = new User(acc, "021", 20000);

        userA.start();
        userB.start();
    }
}
```

Output:

```
Affan is trying to withdraw 45000
Affan is proceeding with withdrawal...
Affan successfully withdrew 45000
Remaining Balance: 5000
021 is trying to withdraw 20000
Insufficient balance for 021 to withdraw 20000
Remaining Balance: 5000
```

2. Create an inter thread communication program of printer job by implementing two threads, one for calculating the remaining pages in printer tray and other one will print the pages that are pending on queue. (Hint: If total pages are 10 and user sends job for 15 pages than print thread will be on wait and will be notified once available pages are equal or greater than printing pages).

```
1 package SDALAB;
2
3 class Printer {
4     int pagesInTray = 10;
5
6     synchronized void printPages(int pagesToPrint) {
7         System.out.println("Print Job requested for " + pagesToPrint + " pages");
8
9         while (pagesInTray < pagesToPrint) {
10             System.out.println("Not enough pages! Current pages: " + pagesInTray);
11             System.out.println("Print thread waiting for pages...");
12             try {
13                 wait();
14             } catch (InterruptedException e) {
15                 System.out.println(e);
16             }
17         }
18
19         pagesInTray -= pagesToPrint;
20         System.out.println("Printing " + pagesToPrint + " pages...");
21         System.out.println("Remaining pages in tray: " + pagesInTray);
22     }
23
24     synchronized void addPages(int pages) {
25         System.out.println("Adding " + pages + " pages to tray...");
26         pagesInTray += pages;
27         System.out.println("Pages after refill: " + pagesInTray);
28         notify();
29     }
30 }
31
32 class PrintJob extends Thread {
33     Printer printer;
```

```
> PrintJob(Printer printer) {
    this.printer = printer;
}

@Override
> public void run() {
    printer.printPages(15);
}

}

class TrayChecker extends Thread {
    Printer printer;

> TrayChecker(Printer printer) {
    this.printer = printer;
}

@Override
> public void run() {
>     try {
        Thread.sleep(3000);
    } catch (InterruptedException e) {}
    printer.addPages(10);
}

}

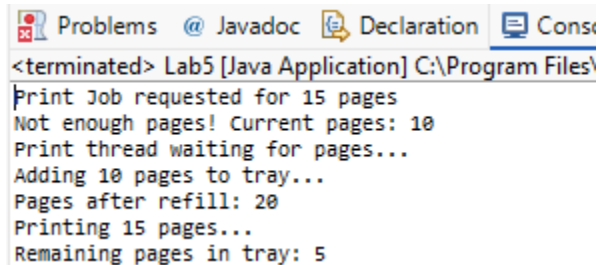
public class Lab5 {
> public static void main(String[] args) {
    Printer printer = new Printer();

    PrintJob printThread = new PrintJob(printer);
    TrayChecker trayThread = new TrayChecker(printer);

    printThread.start();
    trayThread.start();

    }
}
```

Output:



```
<terminated> Lab5 [Java Application] C:\Program Files\
Print Job requested for 15 pages
Not enough pages! Current pages: 10
Print thread waiting for pages...
Adding 10 pages to tray...
Pages after refill: 20
Printing 15 pages...
Remaining pages in tray: 5
```